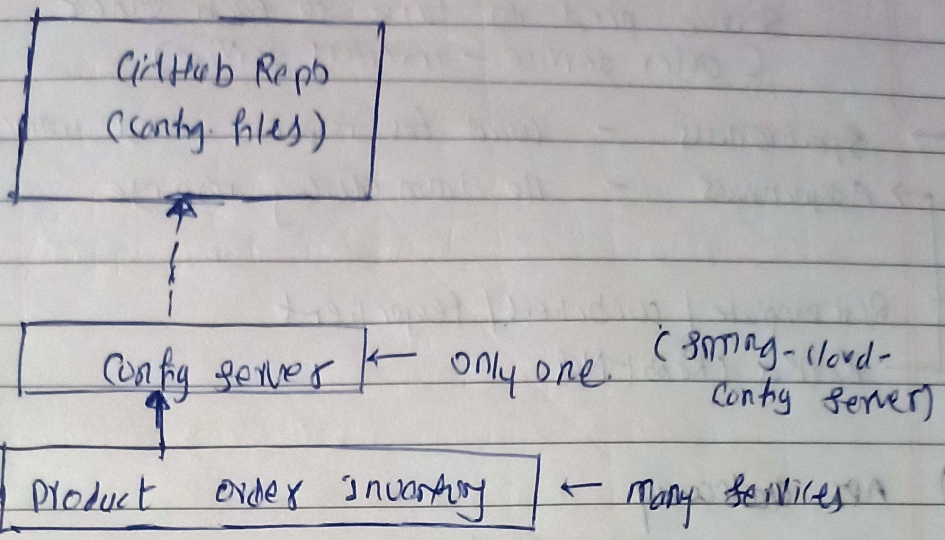
① Spring-cloud-config :-

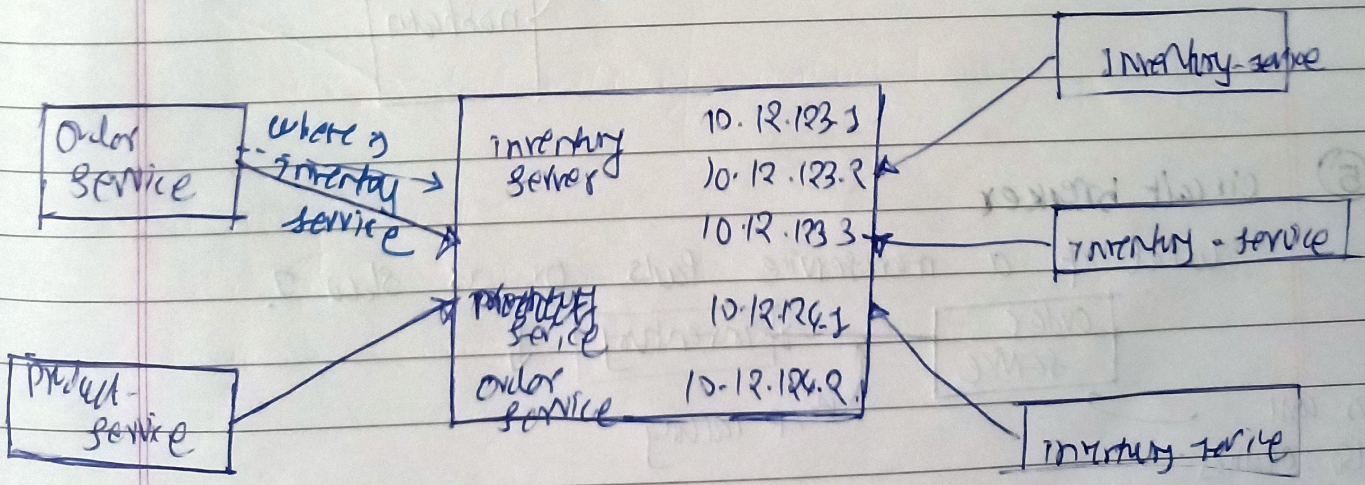


- Managing config files is tricky & hectic task.
- No reload features (Using - Spring-Cloud-Starter-actuator) for live reload.

② Eureka discovery client & server :-

If one microservice want to communicate with other & other microservices has many instances how can ?

- Eureka Discovery client :- Each microservice
- Eureka Discovery Server :- Each microservice register themselves & later can communicate by turning contact of discovery service.



③ Service-to-service communication (inter-process communication).

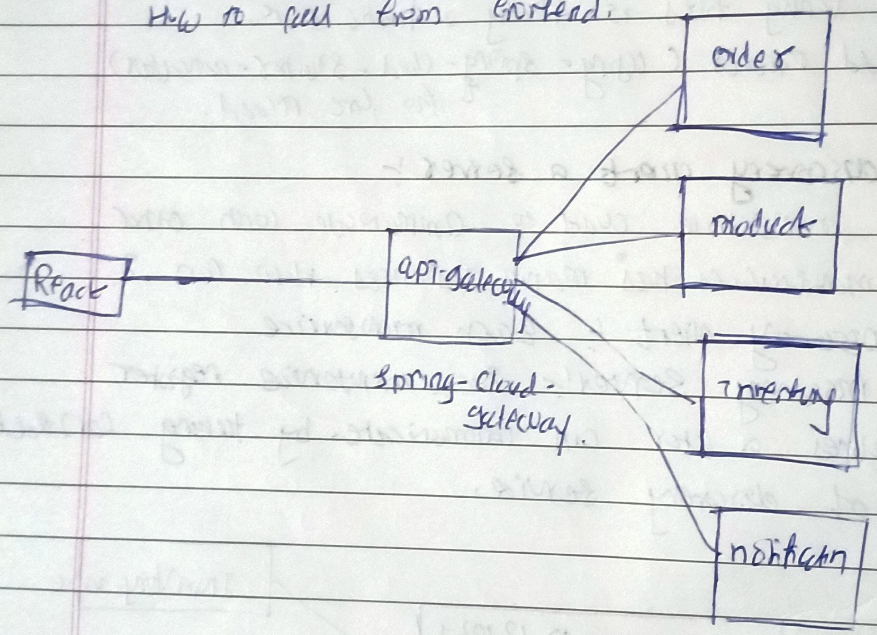
Service need to talk to each other. How?
(Order service - inventory service)

→ Synchronous - wait for response, need immediate response
→ Asynchronous - No immediate response

RestTemplate / webclient / feignclient
Kafka / RabbitMQ

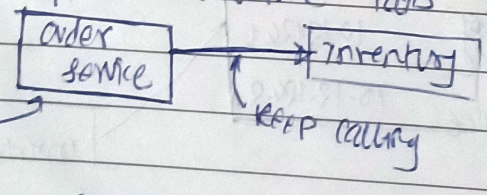
④ API Gateway

Too many entry point (auth-service, order, product etc)
How to call from frontend.



⑤ Circuit breaker

What if a microservice fails or is slow?



cascading failure - one failure spreads other.

Circuit breaker pattern

Resilience4j library

⑥ Distributed Tracing :-

Debugging in Microservices is Hard ?

One user might touch 5 services

(auth → product → inventory - payment - notification)

If something gets wrong

where exactly did fail ?

which service was slow ?

Tools Used :-

Sleuth	add trace & span ID to logs
Zipkin	UI to visualize trace
OpenTelemetry	industry standard API for many of many
Jaegar	another many backend (used with OpenTelemetry)